PROJECT PHASE I REPORT

ON

BROWSEBLOCK

Submitted by

Deon George (SJC23CS086)

Amal Kurian Royce (SJC23CS037)

Maria Royce (SJC23CS159)

Melbin K Thomas (SJC23CS164)

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the degree

of

Bachelor of Technology

in

Computer Science and Engineering



Department of Computer Science and Engineering

St. Joseph's College of Engineering and Technology, Palai

(Autonomous)

October : 2026

Declaration

We undersigned hereby declare that the project phase 1 report on **"BrowseBlock: AI Browser Extension for Real-Time Browser-Based Attack Detection and Prevention"**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Prof. Divya Sunny**. This submission represents our ideas in our own words and where ideas or words of others have been included. We have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Name and Signature of Student

Deon George (SJC23CS086)
Amal Kurian Royce (SJC23CS037)
Maria Royce (SJC23CS159)
Melbin K Thomas (SJC23CS164)

Place: Choondacherry
Date: 17-10-2026

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY, PALAI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the report entitled **"BrowseBlock: AI Browser Extension for Real-Time Browser-Based Attack Detection and Prevention"** submitted by **Deon George (SJC23CS086), Amal Kurian Royce (SJC23CS037), Maria Royce (SJC23CS159), and Melbin K Thomas (SJC23CS164)** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under my guidance and supervision.

Prof. Divya Sunny
Project Guide
Assistant Professor
Department of CSE

Project Coordinator
Prof. Smitha Jacob
Assistant Professor
Department of CSE

Head of Department
Dr. Joby P P
Professor & Head
Department of CSE

Place: Choondacherry
Date: 17-10-2026

Acknowledgement

The success of this Phase I project required the guidance and assistance of many people, and we are privileged to have received their support throughout its completion. We sincerely thank everyone whose supervision, encouragement, and technical assistance made this work possible.

We would like to express our respect and gratitude to the management of St. Joseph's College of Engineering and Technology for providing us with the opportunity and platform to work on this project.

A special word of thanks goes to our beloved Principal, **Dr. V. P. Devassia**, for providing invaluable support and necessary facilities to carry out this project.

We are extremely indebted to **Dr. Joby P. P**, Professor & Head of the Department of Computer Science and Engineering, for his valuable suggestions and encouragement throughout the course of this project work.

We would also like to express our gratitude to our project coordinator, **Prof. Smitha Jacob**, Associate Professor in the Department of Computer Science and Engineering, for her valuable suggestions and guidelines during the entire duration of this project. We are truly appreciate her contributions and technical support in preparing this report.

Our heartfelt thanks go to our project guide, **Prof. Divya Sunny**, Assistant Professor in the Department of Computer Science and Engineering, for her keen interest, guidance, and technical support throughout the project.

We are thankful and fortunate enough to have received constant encouragement, support, and guidance from all the staff members of the Department of Computer Science and Engineering. Their assistance played a crucial role in the successful completion of our Project Phase 1 work.

Abstract

BrowseBlock is an AI/ML-powered browser security platform designed to detect and help prevent browser-based attacks in real time. A Manifest V3 Chromium extension monitors supported navigation, webpage scripts, redirects, frames, notifications, downloads, email content, and observable extension activity. A React and Tailwind CSS dashboard supports authenticated manual scans of URLs, QR codes, emails, and files, together with detection history, threat reports, and explanations. The FastAPI backend coordinates XGBoost URL and file analysis, Random Forest email screening, static security rules, QR decoding, and RAG/LLM-based detailed email analysis. Firebase Authentication verifies user identity and MongoDB stores authorized application records without storing passwords. The proposed system covers URL and fake-QR phishing, cross-site scripting, credential harvesting, malicious redirects, clickjacking, cryptojacking, malicious downloads, email phishing, notification scams, and browser-extension abuse. Each result is classified as Safe, Suspicious, or High-risk and includes evidence, analysis completeness, and the actual protective action. The Phase I work establishes the requirements, architecture, methodology, literature foundation, and evaluation plan for a privacy-conscious security layer that complements existing browser and antivirus protections.

Contents

Declaration ii

Acknowledgement iv

Abstract v

List of Figures viii

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives and Scope . . . 2
1.4 Significance of the Study . . . 2
1.5 Proposed Solution . . . 2
1.6 Expected Outcomes . . . 3

List of Abbreviations 1

2 Literature Review 4

2.1 URL and Fake QR Phishing . . . 4
2.2 HTML and JavaScript Redirects . . . 4
2.3 Malicious File Downloads . . . 4
2.4 Email Phishing . . . 5
2.5 Cryptojacking . . . 5
2.6 Cross-Site Scripting . . . 5
2.7 Clickjacking . . . 6
2.8 Browser Extension Abuse . . . 6
2.9 Browser Notification Scams . . . 6
2.10 Credential Harvesting . . . 7
2.11 Research Gap . . . 7

3 Methodology **8**

3.1 Research and Requirements Method 8

3.2 System Overview 8

3.3 Detection Methodology 8

3.4 Risk and Evidence Model 9

3.5 Privacy and Security Method 9

4 System Study **10**

4.1 Existing System 10

4.2 Proposed System 10

4.3 System Modules 10

4.4 Users and Roles 11

4.5 Constraints 11

5 System Design **12**

5.1 Architecture 12

5.2 Use Case Diagram 12

5.3 Activity Diagram 13

5.4 Sequence Diagram 15

5.5 Data Model 15

6 System Implementation **17**

6.1 Development Environment 17

6.2 Component Integration 17

6.3 Detector Implementation 17

6.4 User Interface and Reports 18

6.5 Testing and Evaluation 18

6.6 Deployment and Operations 18

Conclusion **19**

References **20**

List of Figures

5.1 BrowseBlock use case diagram 13
5.2 BrowseBlock activity diagram 14
5.3 BrowseBlock architecture 15
5.4 BrowseBlock entity relationship diagram 16

Chapter 1

Introduction

1.1 Background

Internet users increasingly encounter threats through ordinary browsing activities: fraudulent URLs, QR-code phishing, deceptive login pages, malicious redirects, injected scripts, unsafe downloads, and abusive browser extensions. Existing browser security features and antivirus products provide important protection, but they do not always explain why a page or action is suspicious, and many threats depend on context that is only visible while the user is browsing. The literature reviewed for this project shows that machine-learning classifiers, static and dynamic analysis, browser instrumentation, and threat-intelligence services can each identify useful indicators, but no single technique provides complete coverage.

1.2 Problem Statement

Users need a lightweight and understandable security layer that can observe supported browser activity, combine multiple indicators, and provide timely warnings without collecting unnecessary private content. Existing approaches are commonly specialized: URL classifiers do not explain page behaviour, blocklists miss new campaigns, and heavyweight analysis cannot run continuously in a browser extension. The problem addressed by BrowseBlock is therefore the design of a unified, explainable system that detects and helps prevent several browser-based attacks while clearly distinguishing a completed protection action from an incomplete or unavailable analysis.

1.3 Objectives and Scope

The project aims to design an AI/ML-powered browser security platform consisting of a Manifest V3 Chromium extension and a React web dashboard backed by FastAPI. Its scope includes URL and fake-QR phishing, cross-site scripting, credential harvesting, HTML/JavaScript redirects, clickjacking, cryptojacking, malicious file downloads, email phishing, browser notification scams, and browser-extension abuse. The system will classify activity as Safe, Suspicious, or High-risk, explain the evidence used, warn or restrict risky activity where browser capabilities permit, and store authorized scan history and reports. It is an additional security layer, not a replacement for antivirus, browser security controls, or enterprise security systems.

1.4 Significance of the Study

BrowseBlock brings threat detection, explanation, and user action into one workflow. General users receive practical warnings, while analysts and organizations can review evidence and historical reports. The design also treats privacy and limitations as security requirements: passwords are never collected for detection, sensitive data is minimized, authentication is handled by Firebase, and MongoDB access is restricted to authorized backend operations.

1.5 Proposed Solution

The extension performs lightweight, event-driven monitoring of navigation, page content, scripts, frames, notifications, downloads, and available extension activity. The FastAPI service coordinates models and deeper analysis, including XGBoost URL and file classification, Random Forest email screening, static file analysis, and RAG/LLM email analysis. The dashboard accepts manual URL, QR, email, and file scans and presents the resulting risk level, indicators, recommended action, and analysis limitations.

1.6 Expected Outcomes

The Phase I outcome is a validated requirements and design foundation for a modular BrowseBlock prototype. It defines the ten detection modules, their interfaces, data flows, security constraints, and evaluation needs. Phase II will implement the selected prototype modules, train and validate models on representative data, test browser compatibility, and measure precision, recall, false positives, latency, and the reliability of warning, blocking, quarantine, and logging actions.

Chapter 2

Literature Review

2.1 URL and Fake QR Phishing

Misiek and Hyla proposed an XGBoost URL phishing detector using URL features and cross-dataset validation. The study reported 90.7% accuracy on a custom dataset and 77.84% average accuracy across three datasets, showing both the efficiency of XGBoost and the difficulty of generalizing to unseen data. BrowseBlock adopts the same direction: URLs are normalized, features such as length, subdomains, IP usage, special characters, and suspicious keywords are extracted, and QR-decoded URLs are passed through the same pipeline. Continuous retraining and evaluation on current datasets are necessary.

2.2 HTML and JavaScript Redirects

Zeng et al.'s "Hidden Path" reconstructs malicious HTTP, JavaScript, and HTML redirect chains using a dynamic Chrome crawler and browser performance logs. It demonstrates that intermediary domains, delayed navigation, fabricated clicks, and iframe techniques can conceal a final destination. BrowseBlock adapts this idea to a lightweight extension by observing supported navigation events, evaluating destinations with the URL detector, and warning or interrupting high-risk navigation. A complete chain cannot be claimed when browser APIs do not expose every event.

2.3 Malicious File Downloads

The Hidost system by Šrndić and Laskov demonstrates static machine-learning detection of malicious PDF and SWF files without executing them. Its structural feature extraction and feature consolidation are relevant to BrowseBlock's download pipeline. BrowseBlock

extends the principle to EXE, ZIP, PDF, DOC, and DOCX files using content-based type identification, format-specific static analysis, SHA-256 hashes, deterministic rules, reputation checks, and XGBoost. Untrusted files must remain unexecuted and analysis must be isolated and resource-limited.

2.4 Email Phishing

Akinyelu and Adewumi classify phishing email with Random Forest using features such as IP-based URLs, link-text mismatches, suspicious keywords, link count, JavaScript, and sender-domain differences. BrowseBlock uses this work for lightweight extension screening, while also incorporating available SPF, DKIM, and DMARC signals. Detailed dashboard analysis adds RAG retrieval, URL extraction, URL-model results, and LLM assessment of urgency, impersonation, social engineering, and credential harvesting. The reported 2014 accuracy cannot be assumed for current attacks; fresh evaluation is required.

2.5 Cryptojacking

Saad and Mohaisen studied more than 5,700 cryptojacking-associated websites using static JavaScript features and dynamic indicators such as CPU consumption, battery drain, and mining-server WebSocket communication. Their results support combining script, WebAssembly, worker, network, and resource-usage evidence. BrowseBlock uses these indicators to warn users or block suspicious resources where supported, while testing against obfuscated miners and legitimate resource-intensive applications.

2.6 Cross-Site Scripting

Pan and Mao combined static analysis with dynamic symbolic execution to detect DOM-sourced XSS in browser-extension scripts, identifying 58 previously unknown vulnerabilities in 154,065 scripts. BrowseBlock applies a lighter event-driven version: AST and pattern checks identify suspicious sources and sinks such as `eval()`, while selective DOM and request monitoring distinguishes reflected, stored, and DOM-based XSS. Suspicious

DOM changes alone do not prove that execution was prevented, so the system must report the actual protective action.

2.7 Clickjacking

ClickProtect detects invisible iframe overlays, deceptive clickable elements, and fake cursors, then reveals elements or warns users. These techniques motivate BrowseBlock's inspection of iframe visibility, positioning, stacking, and interaction structures. Because transparent elements and custom cursors can be legitimate, multiple indicators must be correlated and browser-level controls such as CSP `frame-ancestors` and `X-Frame-Options` should complement extension detection.

2.8 Browser Extension Abuse

CPDetector compares declared Chrome permissions with API usage and host access. Its analysis of 199,058 extensions found excessive permissions in 49.3% of the dataset, illustrating the privacy risk of unnecessary cookies, history, and broad host access. BrowseBlock uses permission mismatches, manifest metadata, webpage tampering, and suspicious script injection as risk indicators. Excessive permission is not by itself proof of malicious behaviour, so dynamic evidence and context remain important.

2.9 Browser Notification Scams

Subramani et al.'s PushAdMiner instrumented Chromium to collect notification permissions, service-worker activity, notification content, network requests, and post-click URLs. Among 21,541 notifications, the study identified 572 campaigns and found 51% of advertisements malicious. BrowseBlock combines notification text, origin, destination URLs, redirects, urgency, fake security alerts, and reward/payment language instead of relying only on blocklists.

2.10 Credential Harvesting

Playwright provides browser automation, JavaScript execution, network observation, locators, and repeatable interaction workflows. BrowseBlock uses it primarily for controlled analysis, testing, and dataset generation: suspicious pages can be loaded with synthetic credentials, login forms and password fields can be inspected, and submission destinations can be compared with the displayed identity. Production monitoring remains lightweight and event-driven, and the system must never collect real passwords or treat Playwright alone as proof of maliciousness.

2.11 Research Gap

The reviewed work provides strong techniques for individual threats but generally focuses on one category, offline analysis, or heavyweight instrumentation. BrowseBlock addresses the integration gap by combining local real-time signals with backend analysis, risk explanations, authorized history, and explicit handling of browser/API limitations. Its evaluation must measure each module independently as well as the combined system, with particular attention to false positives, privacy, latency, and whether an advertised protective action was actually enforced.

Chapter 3

Methodology

3.1 Research and Requirements Method

The project requirements were derived from the BrowseBlock Software Requirements Specification and organized around ten observable browser-threat categories. For each category, the methodology identifies the stimulus, observable evidence, analysis method, risk decision, user response, logging requirements, and browser limitations. The literature review was used to select practical techniques and to avoid transferring reported research accuracy directly to the proposed system.

3.2 System Overview

BrowseBlock is a modular security platform with two client-facing surfaces. A Manifest V3 browser extension provides real-time monitoring and protective feedback. A React and Tailwind CSS dashboard provides authenticated manual scans, detailed reports, analytics, and history. A Python FastAPI backend validates requests, coordinates detectors and AI services, and mediates all database access. Firebase Authentication handles identity, while MongoDB stores authorized application records rather than passwords or authentication secrets.

3.3 Detection Methodology

- **URL and QR analysis**: normalize URLs, extract domain and lexical features, and classify with XGBoost. Decode supported QR codes and analyze their URL payloads.

- **Page and script analysis**: inspect observable scripts, DOM sources and sinks, redirects, frames, overlays, WebAssembly, workers, and notification prompts.
- **Credential and email analysis**: identify deceptive forms and external script domains; use Random Forest for initial email screening and RAG/LLM analysis for authorized detailed investigations.
- **File analysis**: identify file types from content, calculate SHA-256 hashes, extract format-specific static features, apply rules and XGBoost, and never execute untrusted content.
- **Extension analysis**: compare available permissions and host access with observable API use and page changes, treating mismatches as risk indicators.

3.4 Risk and Evidence Model

Every detector produces a normalized result containing a risk level, evidence indicators, analysis completeness, timestamp, and protective action. The levels are Safe, Suspicious, and High-risk. A failed parser, timeout, unsupported format, or unavailable browser event is represented as incomplete analysis rather than a safe verdict. Warnings and blocks are issued only when the relevant browser capability and confidence threshold support the action.

3.5 Privacy and Security Method

Only data required for detection, auditing, and configured retention is collected. Actual passwords are never captured, and email or file content is submitted for deeper analysis only with user authorization. HTTPS protects deployed communication. Firebase ID tokens are verified server-side by FastAPI, authorization is enforced before MongoDB reads or writes, and API credentials and database connections are never exposed to extension or dashboard clients.

Chapter 4

System Study

4.1 Existing System

Current protection is distributed among browser warnings, URL blocklists, antivirus software, email filters, and security extensions. These tools are valuable but often operate independently, provide limited explanations, or cannot inspect all modern browser behaviours. Blocklists can miss new campaigns, while heavyweight crawlers and dynamic analysis are unsuitable for continuous protection on every user page.

4.2 Proposed System

BrowseBlock combines an event-driven extension with a centralized analysis service. The extension observes supported navigation, webpage, download, notification, and extension events. The dashboard enables deliberate scans and detailed review. The backend coordinates model inference, static analysis, RAG/LLM services, authentication, authorization, and persistence. This division keeps common checks lightweight while allowing expensive analysis only when evidence or user intent justifies it.

4.3 System Modules

1. URL and fake-QR phishing detection
2. Cross-site scripting detection and prevention
3. Credential-harvesting detection
4. HTML/JavaScript redirect analysis

5. Clickjacking detection
6. Cryptojacking detection
7. Malicious file-download analysis
8. Email phishing detection
9. Browser notification-scam detection
10. Browser-extension abuse detection

4.4 Users and Roles

General users need concise warnings and practical recommendations. Organizational users need an additional protection layer for browsing, email, and downloads. Authorized security analysts review evidence and threat reports, while administrators manage configuration, services, and access. All role-specific dashboard data is subject to authenticated ownership or explicit authorization.

4.5 Constraints

Detection is bounded by Chromium permissions, browser API visibility, network availability, model quality, external service availability, and the evidence observable at runtime. BrowseBlock therefore complements rather than replaces existing endpoint security. Supported browsers, model versions, datasets, file limits, retention periods, and the first-prototype module set remain implementation decisions to be finalized.

Chapter 5

System Design

5.1 Architecture

The architecture follows four layers. The extension layer observes browser events and provides immediate warnings or supported restrictions. The processing layer contains the FastAPI service and detector orchestration. The intelligence layer contains XGBoost, Random Forest, static-analysis rules, threat-intelligence integrations, and the RAG/LLM workflow. The dashboard and persistence layer provides authenticated interaction, reports, analytics, Firebase identity, and MongoDB records.

5.2 Use Case Diagram

The primary use cases are monitoring browsing activity, scanning a URL or QR code, inspecting a webpage, analyzing a download or email, reviewing evidence, receiving a warning, and viewing authorized history. The user can request a deeper dashboard analysis, while the system administrator manages configuration and authorized analysts review reports.

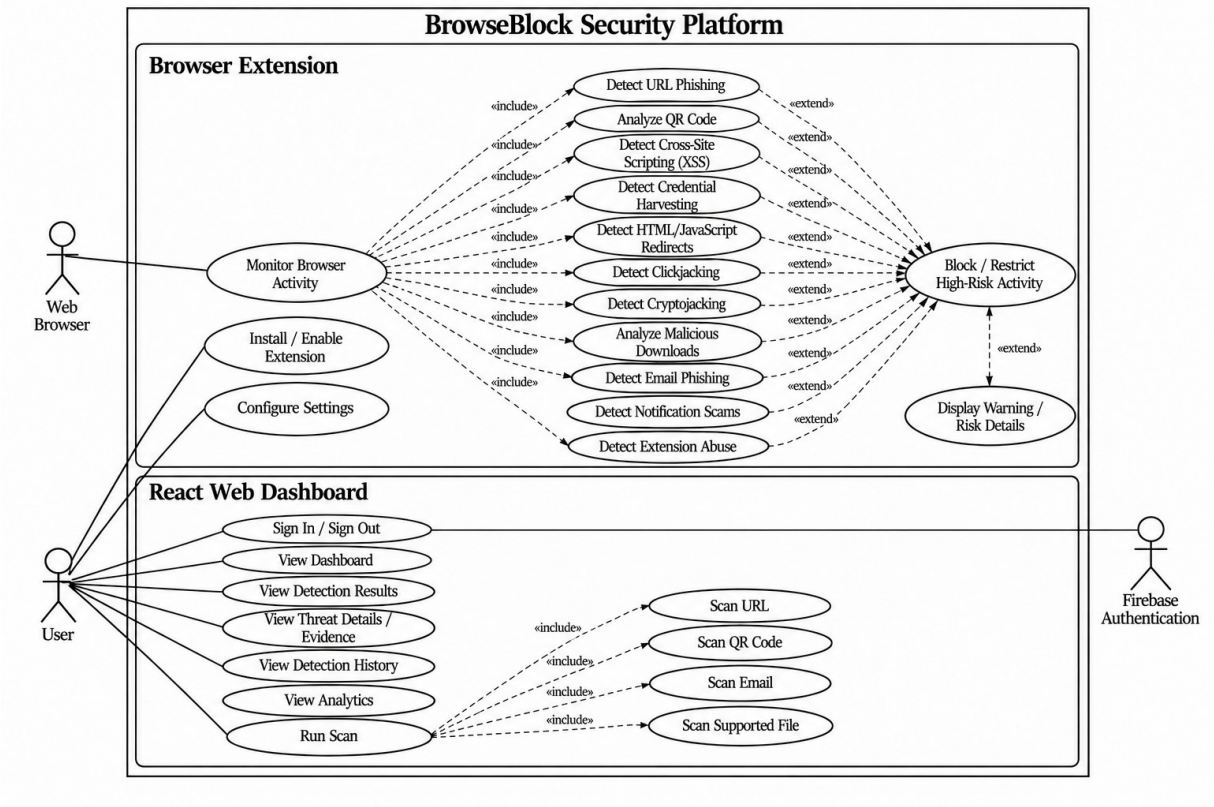


Figure 5.1: BrowseBlock use case diagram

5.3 Activity Diagram

The BrowseBlock workflow begins with a browser event or manual scan. The system validates the input, selects the relevant detector, extracts observable evidence, and performs model and rule-based analysis. It then assigns a risk level, explains the indicators, applies a warning or protective action where supported, and records the result according to privacy and retention policy. Unsupported input, timeout, or incomplete evidence produces an explicit limitation rather than a safe result.

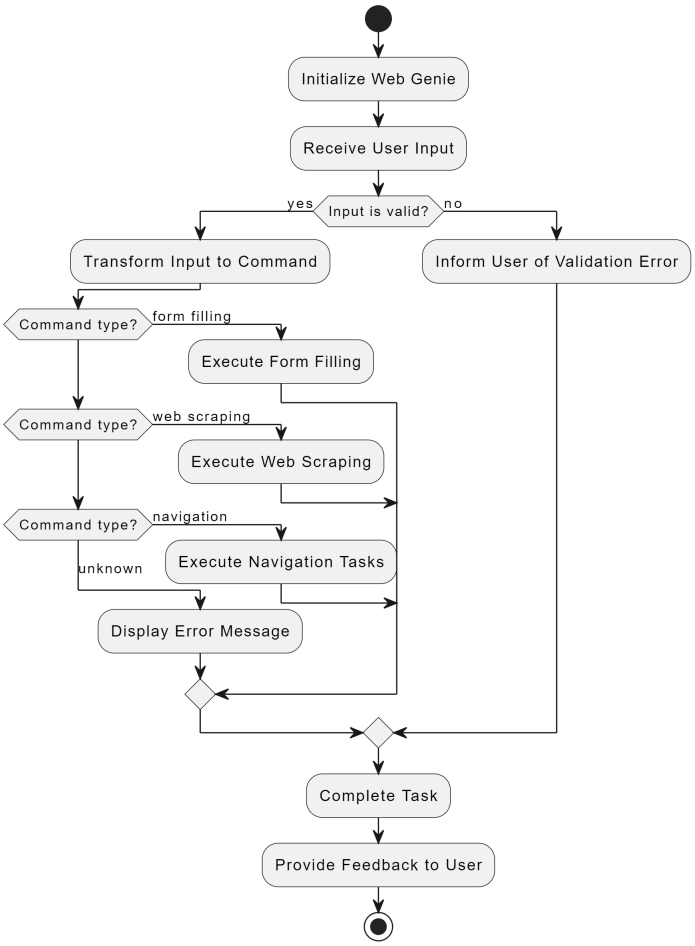


Figure 5.2: BrowseBlock activity diagram

5.4 Sequence Diagram

For a navigation event, the extension captures the URL and available context, sends only the required request to the backend, and receives a risk assessment and evidence. The extension displays the result and applies a supported rule when appropriate. For a dashboard scan, Firebase authenticates the user, FastAPI verifies the ID token, coordinates the selected detector, stores the authorized result in MongoDB, and returns a report. At every stage, failed analysis and unavailable enforcement are reported explicitly.

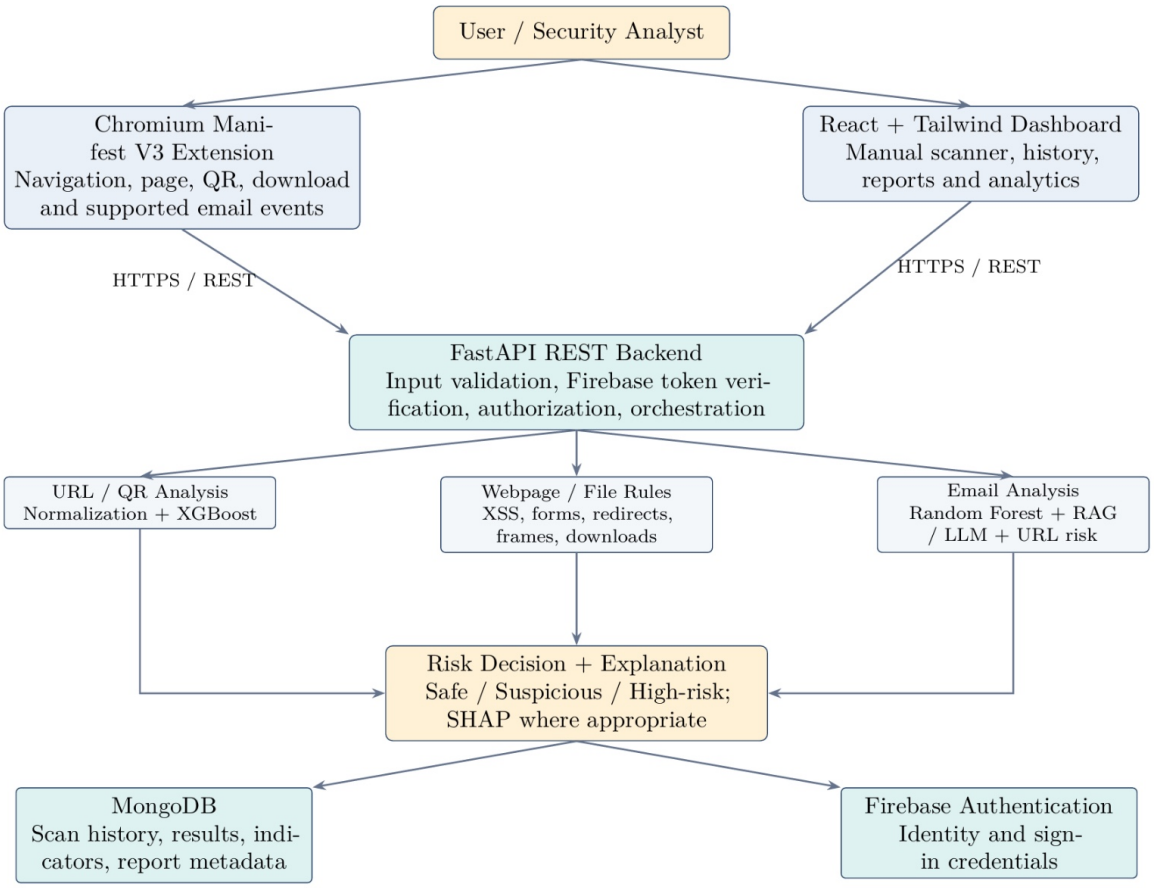


Figure 5.3: BrowseBlock architecture

5.5 Data Model

The data model associates authenticated users with scan requests, detection results, threat reports, and audit records. A result stores the detector category, risk level, indicators,

timestamp, analysis completeness, and protective action. File hashes and minimal metadata may be retained according to policy; raw email, webpage, URL parameters, and file contents are not retained unnecessarily.

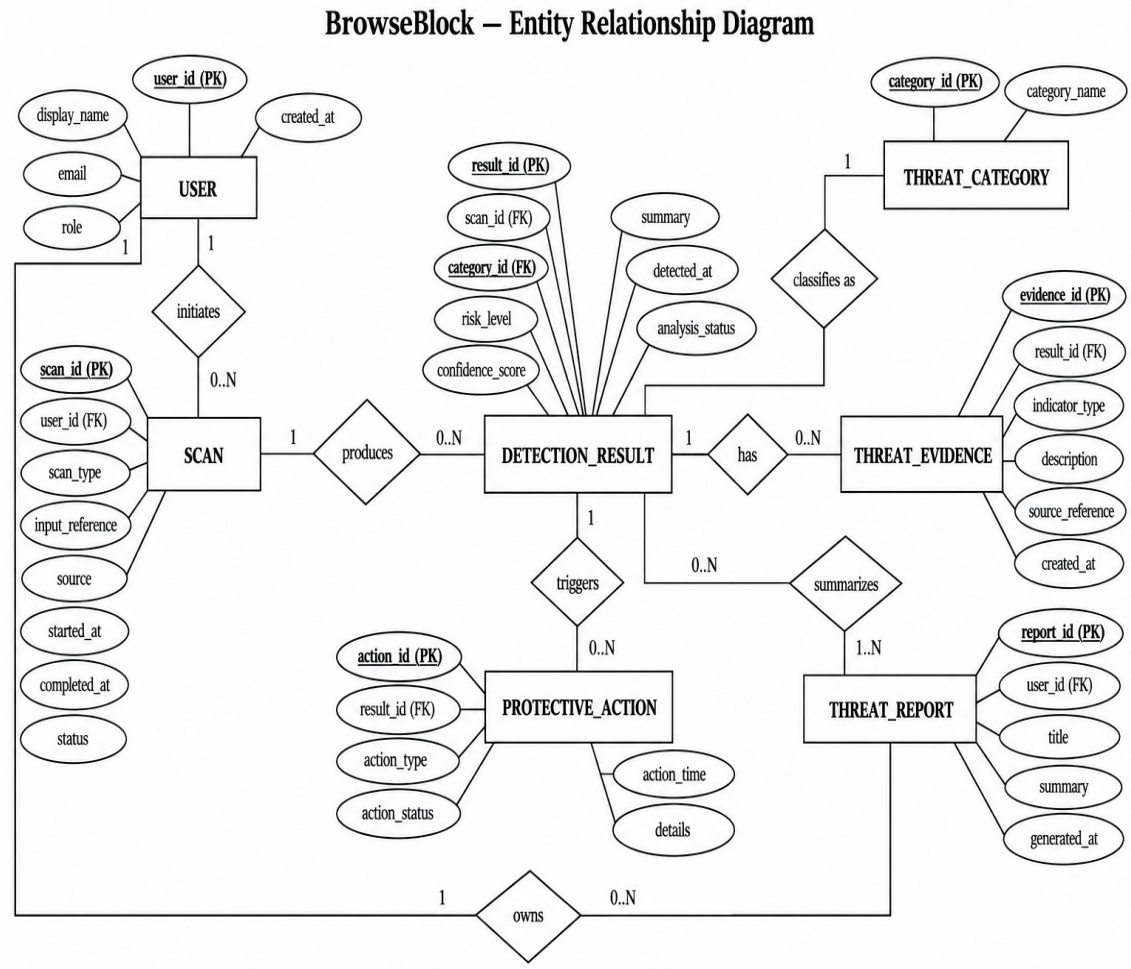


Figure 5.4: BrowseBlock entity relationship diagram

Chapter 6

System Implementation

6.1 Development Environment

The proposed implementation uses a Manifest V3 extension for Chromium-based browsers, React with Tailwind CSS for the dashboard, Python with FastAPI for backend services, Firebase Authentication for identity, and MongoDB for application data. Python machine-learning components use XGBoost for URL and file classification and Random Forest for initial email screening. QR-decoding, file-analysis, Playwright testing, SHAP explanations, and configured RAG/LLM or threat-intelligence services are added where required by the selected prototype modules.

6.2 Component Integration

The extension communicates with FastAPI through HTTPS and sends Firebase ID tokens for protected requests. FastAPI verifies the token, checks authorization, validates the scan request, invokes the appropriate detector, and returns a common result shape. The dashboard uses the same API rather than connecting directly to MongoDB. Detector modules remain independently testable so that models, rules, integrations, and retention policies can be updated without changing the user interface.

6.3 Detector Implementation

URL detection normalizes inputs and extracts configured lexical and domain features. Page detectors use content scripts and supported browser APIs to inspect scripts, redirects, frames, notification prompts, and observable DOM behaviour. Download analysis uses content-based file identification, format-specific parsers, hash and reputation checks,

resource limits, and isolated processing. Email screening is lightweight in the extension and uses RAG/LLM analysis only for an authorized detailed dashboard investigation. Playwright is reserved for controlled analysis and automated testing rather than indiscriminate collection of user credentials.

6.4 User Interface and Reports

The extension presents the risk level, reason, evidence, and available action without overwhelming non-technical users. The dashboard provides login, URL/QR/email/file scan forms, detection history, threat reports, analytics, and troubleshooting information. Reports distinguish detection from prevention and identify incomplete analysis, unsupported formats, timeouts, and unavailable browser capabilities.

6.5 Testing and Evaluation

Unit tests will validate feature extraction, parsers, rules, risk decisions, token verification, authorization, and privacy filtering. Integration tests will cover extension-to-API communication, MongoDB ownership checks, model invocation, and report generation. Browser tests will use safe synthetic pages, controlled phishing fixtures, and synthetic credentials. Evaluation will report precision, recall, false-positive and false-negative rates, latency, resource consumption, model generalization, and confirmed versus requested protective actions.

6.6 Deployment and Operations

The extension and dashboard require configured HTTPS endpoints, Firebase settings, MongoDB access restricted to backend services, model artifacts, and external-service credentials stored outside client code. Logging will contain detection evidence and actions without unnecessary sensitive content. Configuration will define file-size and archive limits, timeouts, retention, supported browsers, and the policy for warning, cancellation, quarantine, or release.

Conclusion

BrowseBlock addresses the need for an explainable, privacy-conscious security layer for browser activity. The requirements and literature study establish a unified architecture consisting of a Manifest V3 extension, React dashboard, FastAPI backend, machine-learning and rule-based detectors, Firebase Authentication, and MongoDB. They also define ten threat categories and clarify that warnings, blocking, quarantine, and analysis completeness must not be overstated.

The Phase I work provides the foundation for implementation by connecting each module to evidence from prior research, explicit functional requirements, interfaces, and evaluation criteria. Phase II will implement and integrate the selected detectors, validate models on representative current data, test browser/API limitations, measure security and performance metrics, and conduct controlled usability testing. BrowseBlock will remain an additional protection layer and will communicate uncertainty wherever the browser or available evidence cannot support a complete decision.

References

[1] M. Misiek and T. Hyla, "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation," 2024.

[2] Y. Zeng, Z. Liu, X. Chen, and T. Zang, "Hidden Path: Understanding the Intermediary in Malicious Redirections," 2022.

[3] N. Šrndić and P. Laskov, "Hidost: A Static Machine-Learning-Based Detector of Malicious Files," 2016.

[4] A. A. Akinyelu and A. O. Adewumi, "Classification of Phishing Email Using Random Forest Machine Learning Technique," 2014.

[5] M. Saad and D. Mohaisen, "Analyzing In-Browser Cryptojacking," 2024.

[6] J. Pan and X. Mao, "Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions," 2017.

[7] D. Pawade, A. Lahigude, D. Reja, and E. Johri, "Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking," 2016.

[8] X. Cheng, Z. Wang, W. Li, W. Zeng, and X. Yang, "CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions," 2025.

[9] K. Subramani, X. Yuan, O. Setayeshfar, P. Vadrevu, K. H. Lee, and R. Perdisci, "When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising," 2020.

[10] Scrapfly, "Web Scraping with Playwright and Python," online documentation.

[11] C. -W. Sung, Y. -K. Lee and Y. -T. Tsai, "A New Pipeline for Generating Instruction Dataset via RAG and Self Fine-Tuning," 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), Osaka, Japan, 2024, pp. 2308-2312, doi: 10.1109/COMPSAC61105.2024.00371.

Department of Computer Science and Engineering, SJCET Palai

[12] A. H. Nasution and A. Onan, "ChatGPT Label: Comparing the Quality of Human-Generated and LLM-Generated Annotations in Low-Resource Language NLP Tasks," in IEEE Access, vol. 12, pp. 71876-71900, 2024, doi: 10.1109/ACCESS.2024.3402809.

[13] H. Kirinuki, S. Matsumoto, Y. Higo and S. Kusumoto, "NLP-assisted Web Element Identification Toward Script-free Testing," 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), Luxembourg, 2021, pp. 639-643, doi: 10.1109/ICSME52107.2021.00072.

[14] Z. Feng, X. Feng, D. Zhao, M. Yang and B. Qin, "Retrieval-Generation Synergy Augmented Large Language Models," ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Seoul, Korea, Republic of, 2024, pp. 11661-11665, doi: 10.1109/ICASSP48485.2024.10448015.

[15] M. Nass, E. Alégroth, R. Feldt, H. SERL, Blekinge Institute of Technology, Karlskrona, Sweden, P. SERL, Blekinge Institute of Technology, Karlskrona, Sweden, and Software Engineering, Chalmers University of Technology, Göteborg, Sweden, "Improving web element localization by using a large language model," Jun. 2024. [Online]. Available: `https://doi.org/10.1002/stvr.1893`.

[16] Y. Sasaki, H. Washizaki, J. Li, D. Sander, N. Yoshioka and Y. Fukazawa, "Systematic Literature Review of Prompt Engineering Patterns in Software Engineering," 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), Osaka, Japan, 2024, pp. 670-675, doi: 10.1109/COMPSAC61105.2024.00096.

[17] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu and B. Myers, "Using an LLM to Help with Code Understanding," 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), Lisbon, Portugal, 2024, pp. 1184-1196, doi: 10.1145/3597503.3639187.

[18] A. Mahalakshmi, N. Sowmya, J. Suvethaa, and S. M. Swaroop, "Web Application Automation using Selenium," International Journal of Innovative Technology and Exploring Engineering, vol. 9, no. 7, pp. 570–573, May 2020, doi: 10.35940/ijitee.f4623.059720.

VISION & MISSION OF THE DEPARTMENT

VISION

- To evolve as a school of computing with globally reputed Centre's of excellence and serve the changing needs of the industry and society.

MISSION

- The department is committed in bringing out career-oriented graduates who are industry ready through innovative practices of teaching and learning process
- To nurture professional approach, leadership qualities and moral values to the graduates by organizing various programs periodically
- To acquire self-sustainability and serve the society through research and consultancy